

**Universidad de la Fuerzas Armadas - ESPE**

**Departamento de Ciencias de la Computación**

**Carrera de Ingeniería de Software**



**Análisis y Diseño de Software - NRC30723**

**Proyecto:** Sistema Bocatto

**Título:** GUÍA 2 PARA DESARROLLAR EN PLANTUML:

“Relación entre casos de uso, diagrama de clases de análisis y diagramas de secuencia de análisis”

**Integrantes:** Carlos Caiza, Kerlly Chiriboga, Alexander Villacres

**Tutor:** Mgtr. Jenny Ruiz

**Fecha de entrega:** 4 de Mayo, 2026

## GUÍA 2 PARA DESARROLLAR EN PLANTUML

### Relación entre casos de uso, diagrama de clases de análisis y diagramas de secuencia de análisis

| Elemento         | Descripción                                                                                                                                                                |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Asignatura       | Análisis y Diseño de Software                                                                                                                                              |
| Tema             | Modelado De Análisis Con Plantuml (Cu0-0 A Cu0-3)                                                                                                                          |
| Caso base        | Gestión de requisitos funcionales según el ERS de Sistema BOCATTO.                                                                                                         |
| Operaciones CRUD | Agregar, Actualizar, Eliminar y Mostrar Todo                                                                                                                               |
| Referencia base  | Guia_PlantUML_CU_Analisis_Grupo2                                                                                                                                           |
| Propósito        | Realizar un análisis estructural y dinámico a partir del análisis funcional previamente realizado, manteniendo coherencia entre requisito, caso de uso, clase y secuencia. |

### 1. Propósito de la guía

Esta guía tiene como finalidad orientar la transición desde el análisis funcional inicial hacia el análisis estructural y dinámico del Sistema Bocatto. El énfasis no radica en la implementación técnica o de interfaz, sino en la comprensión de las responsabilidades de cada objeto bajo el patrón BCE (Boundary-Control-Entity).

Se busca garantizar la trazabilidad integral, asegurando que cada interacción modelada en los diagramas de secuencia sea coherente con los requisitos funcionales de nivel 0 y con las operaciones definidas en el diagrama de clases de análisis.

### 2. Relación general entre los tres modelos

| Modelo                   | Pregunta que responde                                                              | Producto en PlantUML                                                                     | Uso en esta Guía                                                                               |
|--------------------------|------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Casos de uso de análisis | ¿Qué necesita hacer el actor con el sistema?                                       | Diagrama de casos de uso y matriz RF-CU.                                                 | Permite identificar responsabilidades, clases frontera, control y entidad.                     |
| Clases de análisis       | ¿Qué objetos participan para cumplir los casos de uso?                             | Diagrama de clases de análisis con estereotipos boundary, control, entity y repositorio. | Define los objetos (Frontera, Control, Entidad, Repositorio) que aparecerán en las secuencias. |
| Secuencias de análisis   | ¿Cómo colaboran actor, frontera, control y entidad para ejecutar cada caso de uso? | Diagramas de secuencia por operación de pedido del cliente o personal                    | Valida que el flujo dinámico sea coherente con los mensajes declarados en las clases.          |

### 3. Caso base actualizado: Pedidos (Sistema Bocatto)

El caso se basa en interfaces para la toma de pedidos de los clientes, con varios campos de entrada y tres posibles

tipos de pedido.

| Dato                    | Tipo sugerido        | Descripción                               | Validación mínima                     |
|-------------------------|----------------------|-------------------------------------------|---------------------------------------|
| Número de mesa          | Texto corto o entero | Identificador único de la mesa..          | Obligatorio                           |
| Productos seleccionados | Texto                | Producto seleccionado para el pedido      | Obligatorio; por lo menos un producto |
| Cantidad por producto   | Entero               | Cantidad seleccionada de cierto producto. | Obligatoria, numérica y mayor que 0.  |

## 4. Requisitos Funcionales De Nivel 0

Basados en la ERS de Bocatto, se identifican los siguientes requerimientos para el desarrollo de la Guía:

| Código | Requisito Funcional                           | Actor Principal   | Caso de Uso                    |
|--------|-----------------------------------------------|-------------------|--------------------------------|
| RF-00  | Acceso seguro mediante usuario y clave.       | Usuario (General) | CU0-0 Autenticar en el sistema |
| RF-01  | Registro de pedidos directamente en la mesa.  | Mesero            | CU0-1 Tomar pedido en mesa     |
| RF-02  | Toma de pedidos vía telefónica para delivery. | Operador          | CU0-2 Tomar pedido telefónico  |
| RF-03  | Autogestión de pedido mediante código QR.     | Cliente           | CU0-3 Realizar pedido por QR   |

## 5. Diagrama De Casos De Uso De Análisis

Representación del límite del sistema y los actores involucrados en los procesos de pedido.

```
@startuml
left to right direction
skinparam packageStyle rectangle
skinparam shadowing false

' Definición de Actores
actor "Mesero" as M
actor "Operador" as O
actor "Cliente" as C

rectangle "Sistema Bocatto - Gestión de Pedidos" {
    ' Casos de Uso Principales
    usecase "CU0-0: Autenticar en el sistema" as UC0
    usecase "CU0-1: Tomar pedido en mesa" as UC1
    usecase "CU0-2: Tomar pedido telefónico" as UC2
    usecase "CU0-3: Realizar pedido por QR" as UC3

    ' Caso de Uso Común de Validación (Basado en Guía 1)
    usecase "Validar datos del pedido" as UC_Val
}

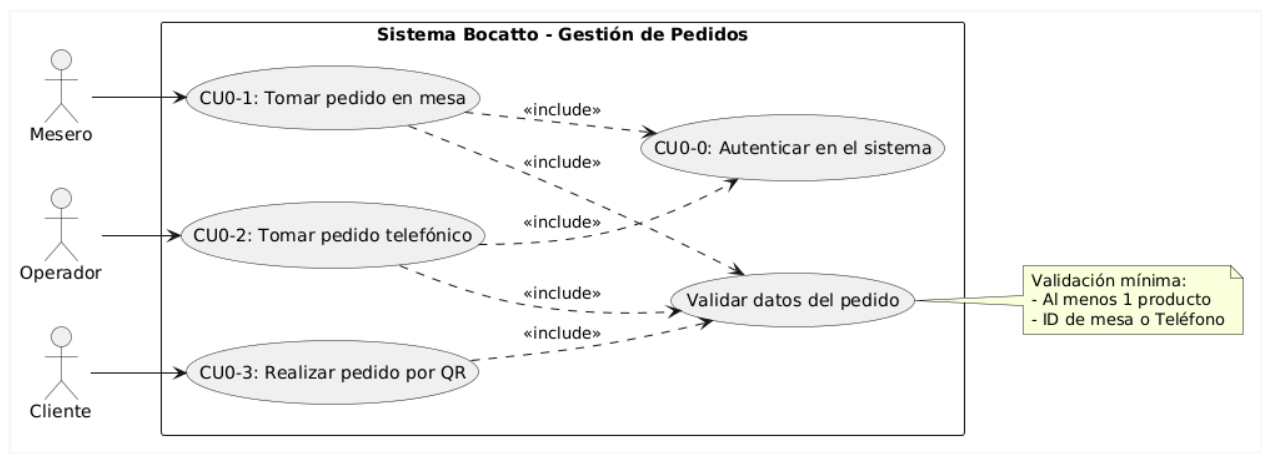
' Relaciones de Actores
M --> UC1
O --> UC2
C --> UC3
```

```
' "El sistema valida que el pedido tenga al menos un producto"
UC1 ..> UC_Val : <<include>>
UC2 ..> UC_Val : <<include>>
UC3 ..> UC_Val : <<include>>
```

```
' El personal interno debe estar autenticado para operar
UC1 ..> UC0 : <<include>>
UC2 ..> UC0 : <<include>>
```

```
note right of UC_Val
Validación mínima:
- Al menos 1 producto
- ID de mesa o Teléfono
end note
```

```
@enduml
```



## 6. Derivación de Clases de Análisis

| Tipo de clase       | Responsabilidad en el análisis               | Clases propuesta       | Relación con los casos de uso                                                               |
|---------------------|----------------------------------------------|------------------------|---------------------------------------------------------------------------------------------|
| Boundary / Frontera | Recibe datos del actor y muestra resultados. | FormularioLogin        | Interfaz para el usuario para acceder al sistema mediante usuario y clave.                  |
|                     |                                              | InterfazPedidoMesa     | Interfaz para el mesero para registrar un pedido para una mesa.                             |
|                     |                                              | InterfazPedidoTel      | Interfaz para el cliente para realizar un pedido por delivery con un número telefónico.     |
|                     |                                              | AppMovilQR             | Interfaz para el cliente para realizar un pedido usando un código QR localizado en la mesa. |
| Control             | Coordina la lógica de cada caso de uso.      | GestorAutenticacion    | Valida que las credenciales ingresadas sean válidas.                                        |
|                     |                                              | ControlRegistroPedidos | Administra los pedidos registrados por los usuarios.                                        |

|                         |                                        |                     |                                                                                                           |
|-------------------------|----------------------------------------|---------------------|-----------------------------------------------------------------------------------------------------------|
| Entity / Entidad        | Representa la información del dominio. | Usuario             | Superentidad que incluye a los clientes, meseros y operadores.                                            |
|                         |                                        | Pedido              | Conjunto de platos solicitados por los clientes.                                                          |
|                         |                                        | Plato               | Entidad que representa uno de los diferentes platos disponibles.                                          |
|                         |                                        | Cliente             | Entidad que realiza pedidos.                                                                              |
| Repositorio / Colección | Conserva y recupera registros.         | RepositorioUsuarios | Permite validar las credenciales de los usuarios para brindar acceso a cada uno a su respectiva interfaz. |

```

@startuml
skinparam classAttributeIconSize 0
skinparam shadowing false
left to right direction

class "FormularioLogin" <<boundary>> {
    + ingresarCredenciales(user, pass)
    + mostrarMenu()
}

class "InterfazPedidoMesa" <<boundary>> {
    - txtMesa: String
    + seleccionarMesaYPlatos(mesa, lista)
    + mostrarExito()
}

class "InterfazPedidoTel" <<boundary>> {
    - txtTelefono: String
    + ingresarTelefono(tel)
    + cargarPedido(lista)
    + mostrarDatosCliente()
}

class "AppMovilQR" <<boundary>> {
    + escanearQR(mesaID)
    + confirmarCarrito(items)
    + mostrarMenuDigital()
}

class "GestorAutenticacion" <<control>> {
    + validarAcceso(user, pass): boolean
    + identificarRol(user): Rol
}

class "ControlRegistroPedido" <<control>> {
    + validarDatos(lista): boolean
    + procesarNuevoPedido(mesa, lista)
    + validarCliente(tel): Cliente
    + obtenerMenu(): List<Plato>
    + enviarOrden(items)
}

class "Usuario" <<entity>> {
    - user: String
    - pass: String
    - rol: String
    + compararClave(pass): boolean
}

```

```

class "Pedido" <<entity>> {
    - id: String
    - mesa: String
    - estado: String
    + Pedido(mesa)
}

class "Plato" <<entity>> {
    - nombre: String
    - precio: double
    + fetchAll(): List<Plato>
}

class "Cliente" <<entity>> {
    - telefono: String
    - direccion: String
    + obtenerDatos(): Cliente
}

class "RepositorioUsuarios" <<repository>> {
    + buscarUsuario(user): Usuario
}

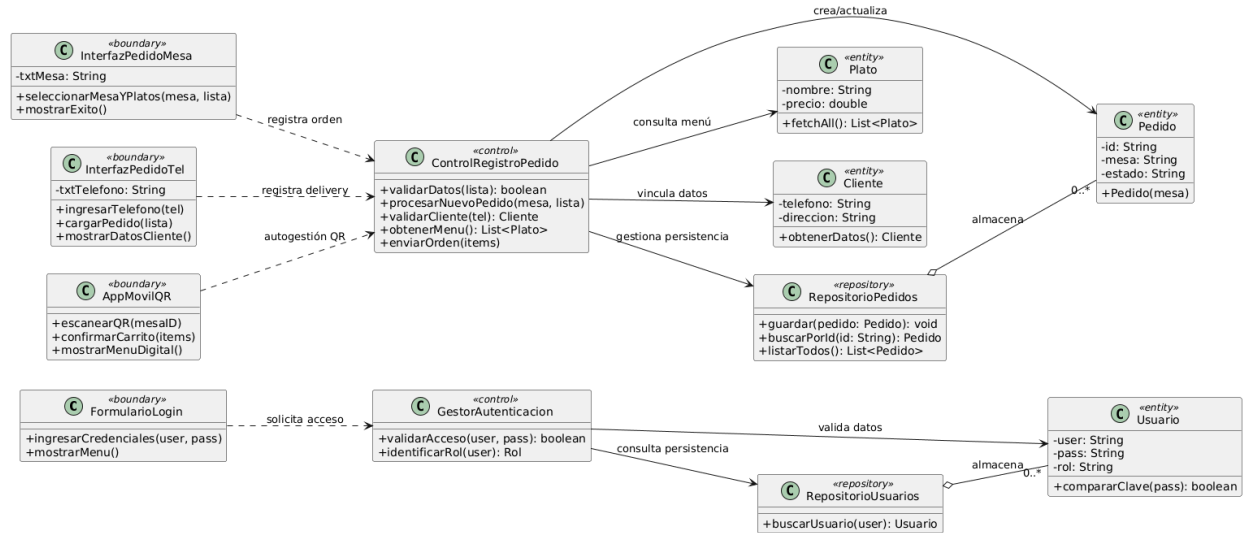
class "RepositorioPedidos" <<repository>> {
    + guardar(pedido: Pedido): void
    + buscarPorId(id: String): Pedido
    + listarTodos(): List<Pedido>
}

FormularioLogin ..> GestorAutenticacion : solicita acceso
InterfazPedidoMesa ..> ControlRegistroPedido : registra orden
InterfazPedidoTel ..> ControlRegistroPedido : registra delivery
AppMovilQR ..> ControlRegistroPedido : autogestión QR

GestorAutenticacion --> Usuario : valida datos
GestorAutenticacion --> RepositorioUsuarios : consulta persistencia
ControlRegistroPedido --> Pedido : crea/actualiza
ControlRegistroPedido --> Plato : consulta menú
ControlRegistroPedido --> Cliente : vincula datos
ControlRegistroPedido --> RepositorioPedidos : gestiona persistencia

RepositorioUsuarios o-- "0..*" Usuario : almacena
RepositorioPedidos o-- "0..*" Pedido : almacena
@enduml

```



## 7. Matriz De Relación: Casos De Uso, Clases Y Secuencias

| Caso de uso                                      | Boundary                                                             | Control                                                                     | Entity / Repositorio                                                                          | Diagrama de secuencia esperado                                                                               |
|--------------------------------------------------|----------------------------------------------------------------------|-----------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| <b>CU0-0:</b><br><b>Autenticar en el Sistema</b> | FormularioLogin: Captura usuario y contraseña.                       | GestorAutenticacion : Coordina la validación de acceso y asignación de rol. | Usuario / RepositorioUsuarios: Verifica credenciales en la colección de usuarios.             | Ingresar datos → buscar usuario → comparar clave → identificar rol → habilitar acceso.                       |
| <b>CU0-1:</b><br><b>Tomar pedido en mesa</b>     | InterfazPedidoMesa: Captura mesa, platos y observaciones.            | ControlRegistroPedido: Administra pedidos e incluye la validación de datos. | Pedido, Plato, Mesero / RepositorioPedidos: Objetos para la creación y guardado del registro. | Autenticación previa → Ingresar mesa → Seleccionar productos → Validar (mín. 1 plato) → Registrar → Guardar. |
| <b>CU0-2:</b><br><b>Tomar pedido telefónico</b>  | InterfazPedidoTel: Captura datos de contacto, dirección y productos. | ControlRegistroPedido: Administra pedidos e incluye la validación de datos. | Cliente, Pedido, Plato / RepositorioPedidos: Vincula al cliente con su orden.                 | Autenticación previa → Validar cliente → Seleccionar → Validar datos → Registrar pedido → Guardar.           |
| <b>CU0-3:</b><br><b>Realizar pedido por QR</b>   | AppMovilQR: Identifica mesa mediante QR y captura productos.         | ControlRegistroPedido: Administra pedidos e incluye la validación de datos. | Pedido, Plato, Cliente / RepositorioPedidos: Entidades para la autogestión.                   | Escanear QR → Mostrar menú → Seleccionar productos → Validar (mín. 1 plato) → Confirmar → Guardar.           |

## 8. Diagramas De Secuencia De Análisis

### 8.1 CU0-0: Autenticar en el sistema

@startuml

```

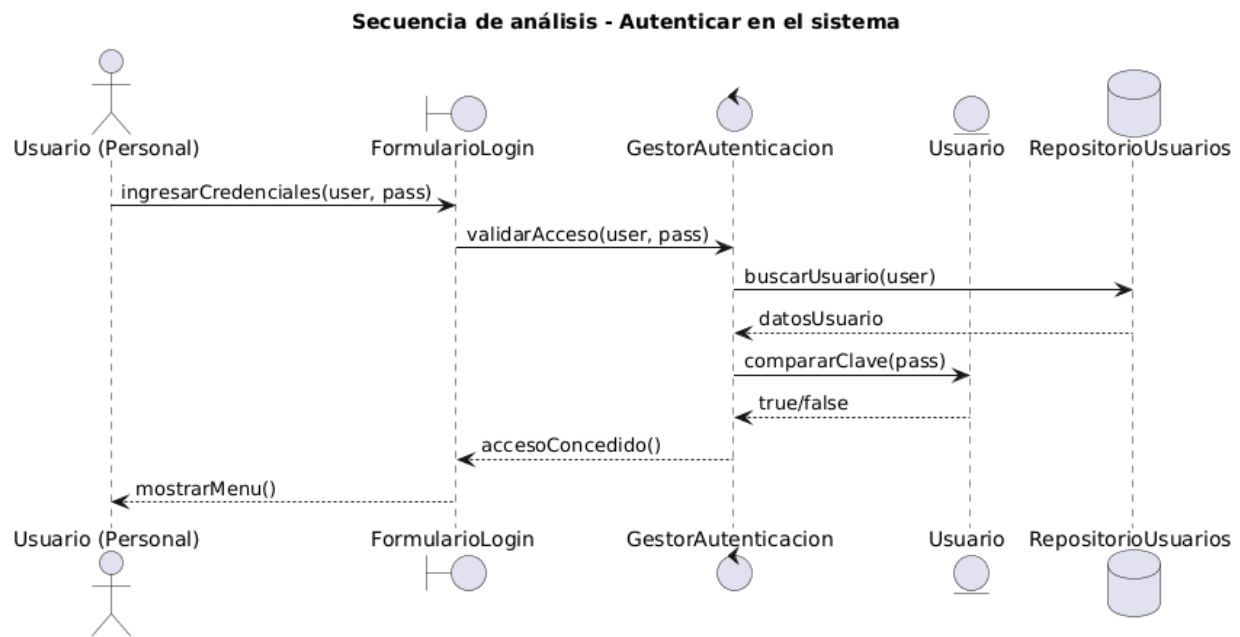
title Secuencia de análisis - Autenticar en el sistema
actor "Usuario (Personal)" as Actor
boundary "FormularioLogin" as UI
control "GestorAutenticacion" as Ctrl
entity "Usuario" as Ent
database "RepositorioUsuarios" as Repo

```

```

Actor -> UI : ingresarCredenciales(user, pass)
UI -> Ctrl : validarAcceso(user, pass)
Ctrl -> Repo : buscarUsuario(user)
Repo --> Ctrl : datosUsuario
Ctrl -> Ent : compararClave(pass)
Ent --> Ctrl : true/false
Ctrl --> UI : accesoConcedido()
UI --> Actor : mostrarMenu()
@enduml

```



## 8.2 CU0-1: Tomar pedido en mesa

```

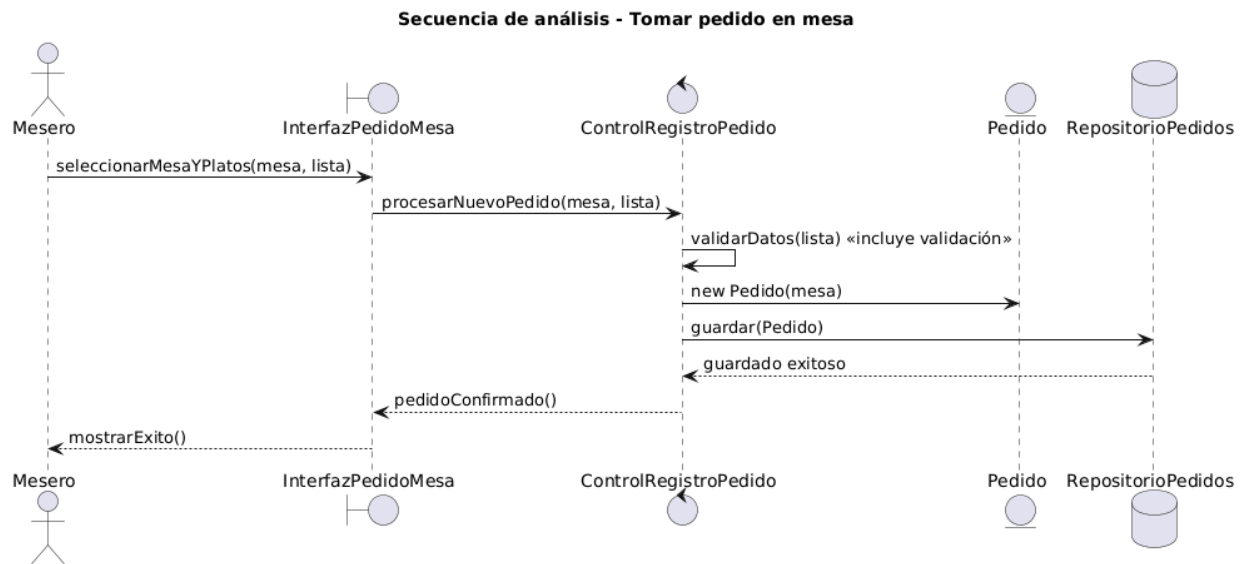
@startuml
title Secuencia de análisis - Tomar pedido en mesa
actor Mesero as Actor
boundary InterfazPedidoMesa as UI
control ControlRegistroPedido as Ctrl
entity Pedido as Ent
database RepositorioPedidos as Repo

```

```

Actor -> UI : seleccionarMesaYPlatos(mesa, lista)
UI -> Ctrl : procesarNuevoPedido(mesa, lista)
Ctrl -> Ctrl : validarDatos(lista) <<incluye validación>>
Ctrl -> Ent : new Pedido(mesa)
Ctrl -> Repo : guardar(Pedido)
Repo --> Ctrl : guardado exitoso
Ctrl --> UI : pedidoConfirmado()
UI --> Actor : mostrarExito()
@enduml

```



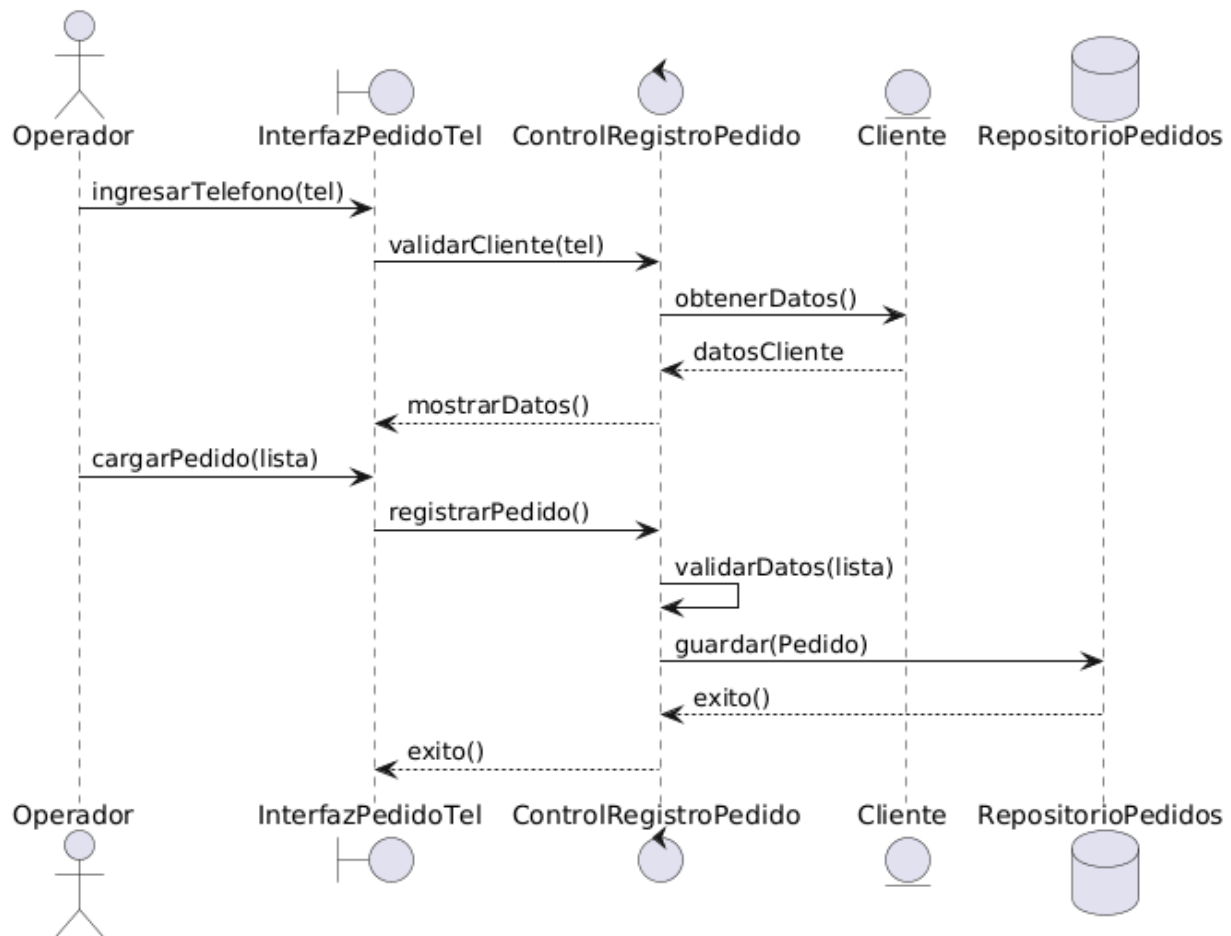
### 8.3 CU0-2: Tomar pedido telefónico

```

@startuml
title Secuencia de análisis - Tomar pedido telefónico
actor Operador as Actor
boundary InterfazPedidoTel as UI
control ControlRegistroPedido as Ctrl
entity Cliente as Ent
database RepositorioPedidos as Repo

Actor -> UI : ingresarTelefono(tel)
UI -> Ctrl : validarCliente(tel)
Ctrl -> Ent : obtenerDatos()
Ent --> Ctrl : datosCliente
Ctrl --> UI : mostrarDatos()
Actor -> UI : cargarPedido(lista)
UI -> Ctrl : registrarPedido()
Ctrl -> Ctrl : validarDatos(lista)
Ctrl -> Repo : guardar(Pedido)
Repo --> Ctrl : exito()
Ctrl --> UI : exito()
@enduml
    
```

### Secuencia de análisis - Tomar pedido telefónico

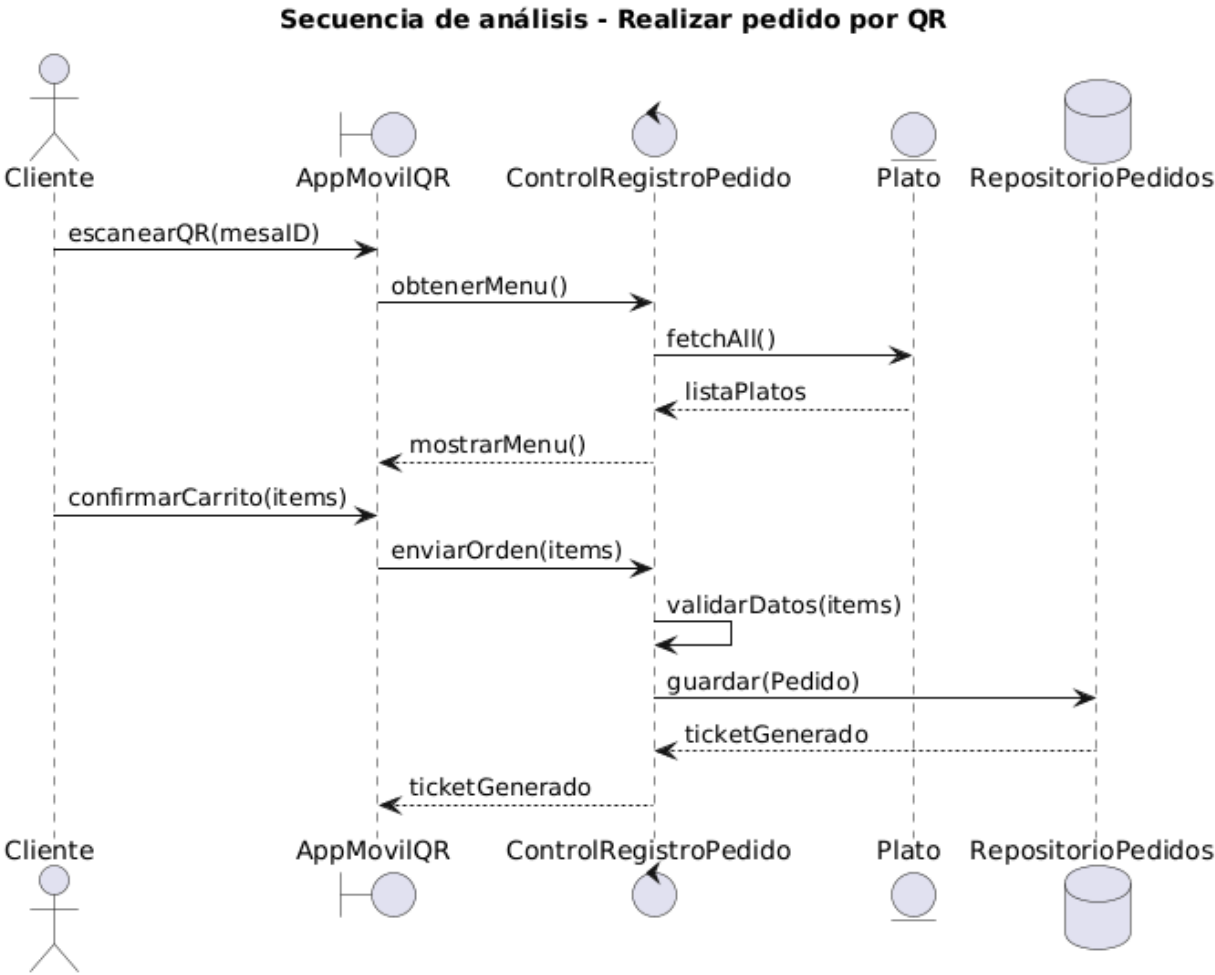


### 8.4 CU0-3: Realizar pedido por QR

```

@startuml
title Secuencia de análisis - Realizar pedido por QR
actor Cliente as Actor
boundary AppMovilQR as UI
control ControlRegistroPedido as Ctrl
entity Plato as Ent
database RepositorioPedidos as Repo

Actor -> UI : escanearQR(mesaID)
UI -> Ctrl : obtenerMenu()
Ctrl -> Ent : fetchAll()
Ent --> Ctrl : listaPlatos
Ctrl --> UI : mostrarMenu()
Actor -> UI : confirmarCarrito(items)
UI -> Ctrl : enviarOrden(items)
Ctrl -> Ctrl : validarDatos(items)
Ctrl -> Repo : guardar(Pedido)
Repo --> Ctrl : ticketGenerado
Ctrl --> UI : ticketGenerado
@enduml
    
```



9. Matriz De Trazabilidad (Rf - Cu - Clases)

| RF    | Caso de Uso                     | Objetos (BCE + Repositorio)                                                          | Prioridad |
|-------|---------------------------------|--------------------------------------------------------------------------------------|-----------|
| RF-00 | CU0-0: Autenticar en el sistema | FormularioLogin, GestorAutenticacion, Usuario, RepositorioUsuarios                   | Alta      |
| RF-01 | CU0-1: Tomar pedido en mesa     | InterfazPedidoMesa, ControlRegistroPedido, Pedido, Plato, RepositorioPedidos         | Alta      |
| RF-02 | CU0-2: Tomar pedido telefónico  | InterfazPedidoTel, ControlRegistroPedido, Cliente, Pedido, Plato, RepositorioPedidos | Media     |

| RF    | Caso de Uso                   | Objetos (BCE + Repositorio)                                                   | Prioridad |
|-------|-------------------------------|-------------------------------------------------------------------------------|-----------|
| RF-03 | CU0-3: Realizar pedido por QR | AppMovilQR, ControlRegistroPedido, Pedido, Plato, Cliente, RepositorioPedidos | Alta      |

## 10. Especificación breve de casos de uso conectada con las secuencias

| Caso de uso                            | Precondición                                                                      | Flujo principal resumido                                                                                               | Resultado esperado                                                            |
|----------------------------------------|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| <b>CU0-0: Autenticar en el sistema</b> | El usuario debe estar registrado previamente con un rol asignado.                 | Ingresar credenciales -> validar acceso -> identificar rol -> habilitar funcionalidades.                               | Acceso concedido y menú mostrado según el rol del usuario.                    |
| <b>CU0-1: Tomar pedido en mesa</b>     | El mesero debe haber iniciado sesión correctamente en el sistema.                 | Seleccionar mesa y platos -> validar datos (mín. 1 producto) -> registrar orden -> generar ID -> enviar a cocina.      | Pedido registrado con estado "pendiente" y enviado a cocina para preparación. |
| <b>CU0-2: Tomar pedido telefónico</b>  | El operador debe estar autenticado en su estación de trabajo.                     | Ingresar teléfono del cliente -> validar cliente -> seleccionar productos -> validar datos -> registrar pedido.        | Pedido para delivery registrado correctamente con estado inicial "pendiente". |
| <b>CU0-3: Realizar pedido por QR</b>   | El cliente debe contar con un dispositivo móvil con cámara y conexión a internet. | Escanear código QR -> ver menú digital -> seleccionar productos -> validar datos -> confirmar carrito -> enviar orden. | Pedido autogestionado correctamente, asociado a la mesa y visible en cocina.  |

## 11. Conclusión Técnica

El modelo de análisis desarrollado para el Sistema Bocatto garantiza una coherencia técnica integral, ya que asegura que los requisitos funcionales (“qué”) se transformen en responsabilidades estructurales mediante el patrón BCE (“quiénes”) y se validen a través de diagramas de secuencia (“cómo”). La principal utilidad de este enfoque radica en la trazabilidad estricta: cada mensaje modelado en las secuencias dinámicas (como validarDatos o guardar) corresponde exactamente a una operación declarada en las clases de análisis y en la capa de persistencia de los Repositorios, eliminando ambigüedades antes de la implementación.

Esta versión del modelo refuerza la robustez del sistema al integrar la lógica de validación obligatoria (mínimo un producto) como un comportamiento incluido (<<include>>) y al establecer una dependencia de autenticación para el personal interno, asegurando que las funciones críticas no operen como piezas aisladas. La inclusión de Repositorios cierra el ciclo de análisis al definir cómo se recupera y conserva la información del dominio.

En conclusión, el uso de PlantUML como herramienta de “Diagram as Code” permitió verificar que el sistema no solo es funcionalmente completo, sino estructuralmente sólido y modular. Esto facilita su futura escalabilidad y mantenimiento al centralizar la lógica de negocio en controladores compartidos y entidades bien definidas, cumpliendo con los estándares de calidad exigidos en la ruta de análisis.

